CIRCUIT MERGE

Assemblez l’informatique

Un jeu Python de chute, de collisions et de fusion.
Une base documentée pour un projet de NSI.



MANUEL DE PRISE EN MAIN · GUIDE DU CODE · SUJET DE PROJET

Code commenté en français, onze composants, mode pédagogique, exercices et tests.

6 octobre 2026 • Python 3.12 / Pygame 2.6.1 / Pymunk 7.3.0

Prise en main

1. Extraire entièrement le ZIP. 2. Installer Python 3.12 si nécessaire. 3. Sous Windows, exécuter **Installer_Windows.bat** puis **Lancer_Windows.bat**. L'installation télécharge les dépendances ; les parties suivantes fonctionnent hors ligne.

```
python -m pip install -r requirements.txt
python main.py
```

Sur macOS ou Linux, utiliser un environnement virtuel Python comme indiqué dans le README. Le jeu vise les ordinateurs ; aucune application Android n'est incluse.

Les commandes

Action	Commande
Viser	Souris, flèches ou Q/D
Lâcher	Clic puis relâcher / Espace
Pause	P / Échap
Recommencer	R
Mode NSI / ralenti	F1 / F2
Son / plein écran	M / F11
Inspecteur	Survol ou clic droit en mode NSI

Les règles

Deux composants identiques qui se touchent deviennent le suivant, plus grand. Les cinq petits niveaux peuvent être lâchés ; les suivants sont obtenus par fusion. Le score vient des fusions. Une pièce dépassant la limite durant 2 secondes continues termine la partie, après 1,15 seconde de tolérance suivant sa création.

Créer un supercalculateur est l'objectif. La partie continue ; deux supercalculateurs se retirent et libèrent le bac. Le prochain composant est annoncé à droite. Le record et la partie sont sauvegardés automatiquement.

La progression est ludique : deux résistances ne deviennent pas réellement une LED. Les paramètres sont réglés pour Circuit Merge et ne prétendent pas reproduire les constantes internes du jeu Android.

Repères pour le projet

Document / dossier	Utilité
README.md	Installation, lancement, règles et options
docs/GUIDE_NSI.md	Architecture et lectures guidées
docs/SUJET_PROJET_NSI.md	Consignes distribuables et barème
docs/NOTES_ENSEIGNANT.md	Pistes de correction et différenciation
atelier_nsi/	Fonctions élèves sans dépendance
experiences/	Mesure de l’erreur numérique d’une chute
tests/	Scénarios de physique, d’interface et de sauvegarde

Lecture recommandée

Jouer → décrire les règles → config → model → engine → app → view. La version complète est une référence ; on peut distribuer seulement le sujet et le squelette pour laisser les élèves construire leurs propres règles.

Les trois couches

Couche	Ce qu’elle fait
Moteur	Stocke les pièces, simule les contacts et applique les règles
Vue	Dessine les composants et les animations
Application	Lit les entrées et organise le temps / la sauvegarde

Le catalogue

Niveau	Composant	Rayon / points
01	Résistance	17 px / 1
02	LED	24 px / 3
03	Condensateur	31 px / 6
04	Transistor	39 px / 10
05	Puce logique	48 px / 15
06	Microcontrôleur	59 px / 21
07	Mémoire RAM	71 px / 28
08	Processeur	84 px / 36
09	Carte graphique	98 px / 45
10	Carte mère	114 px / 55
11	Supercalculateur	132 px / 66

Guide NSI — comprendre Circuit Merge

Ce jeu est à la fois un programme complet et un terrain d'expérimentation. L'objectif pédagogique est d'expliquer les choix, d'établir des contrats puis de modifier un comportement en s'appuyant sur des tests.

1. Faire une première lecture utile

Commencer par jouer cinq minutes. Décrire les règles sans lire le code : quand la fusion se produit-elle ? Quel objet apparaît ensuite ? Que signifie « dépasser la limite » ? Une collision est-elle forcément une fusion ?

Ouvrir ensuite les fichiers dans cet ordre :

1. `config.py` : quels paramètres permettent de décrire les règles ?
2. `model.py` : quelles informations faut-il conserver pour une pièce ?
3. `engine.py`, méthodes `drop`, `_on_contact`, `_process_merges`, `_check_overflow` : où les règles deviennent-elles des algorithmes ?
4. `app.py`, méthode `update` : comment le temps fait-il avancer le jeu ?
5. `view.py` : comment un état devient-il une image ?

Pour la première NSI, partir des fonctions du squelette élèves et des collections. Pour la terminale, approfondir les objets, les responsabilités, les événements et les invariants. La simulation des contacts est confiée à Pymunk : construire un moteur complet de corps rigides n'est pas nécessaire pour réaliser le projet.

2. Séparer règles, affichage et application

Module	Responsabilité	Exemple de question
`config`	Paramètres et catalogue	Quel rayon associer au niveau 4 ?
`model`	Données et états	Quels champs distinguent deux résistances ?
`engine`	Règles et simulation	Quels contacts donnent une fusion ?
`app`	Entrées et orchestration	Un clic doit-il viser ou ouvrir un menu ?
`view` / `art`	Rendu et décorations	Comment dessiner une carte graphique ?
`audio`	Synthèse sonore	Quelle fréquence associer à une fusion ?
`storage`	Persistance	Comment retrouver une partie interrompue ?

Le moteur n'importe jamais Pygame. On peut donc le faire évoluer dans un test ou une expérience sans créer de fenêtre. La vue lit les pièces et reçoit des événements ; elle ne décide pas des fusions.

Cette séparation réduit les dépendances : remplacer les dessins ne modifie pas la gravité ; ajouter un test de score ne nécessite pas de carte son.

3. Modéliser les données

`ComponentSpec` est une dataclass **immuable** : elle décrit un type de composant (nom, rayon, couleur, points, rôle). Le catalogue est un tuple.

`Piece` est une dataclass **mutable** : elle décrit une instance dans la partie. Elle contient un identifiant unique, un niveau, un corps, une forme, une date de naissance et son chronomètre de dépassement.

Il faut distinguer trois notions :

- le **niveau** : indice de 0 à 10 dans le catalogue ;
- le **corps** Pymunk : position, vitesse, angle, masse et moment d'inertie ;
- la **forme** Pymunk : cercle utilisé pour calculer les contacts.

Deux résistances ont le même niveau mais des corps, des positions et des identifiants différents. Le sprite n'est qu'une représentation : les trous entre les pattes d'un transistor ne sont pas des zones de collision.

Le dictionnaire `pieces` associe un identifiant à une pièce. `_by_shape` permet de retrouver une pièce depuis une forme reçue dans un contact. Les deux index doivent rester cohérents : ajouter ou supprimer une pièce implique de mettre à jour les deux index et l'espace Pymunk.

4. Comprendre la boucle de jeu

Une image suit cet ordre : lire les événements clavier / souris ; mesurer le temps écoulé ; réaliser les pas de simulation nécessaires ; consommer les événements de jeu ; dessiner ; afficher.

Un écran rapide peut afficher 144 images par seconde et un écran lent 30. Si les pièces avançaient d'une distance fixe par image, la vitesse dépendrait du matériel. Il faut exprimer les vitesses en pixels par seconde.

La simulation utilise un **pas fixe de 1/120 seconde**. L'accumulateur garde le temps réel non encore simulé :

```
accumulateur += temps_ecoule
while accumulateur >= pas_fixe:
    moteur.step(pas_fixe)
    accumulateur -= pas_fixe
```

À 60 images par seconde, deux pas de physique sont normalement réalisés par image. À 30 images par seconde, il y en a quatre. La version réelle plafonne le temps d'une image à 0,1 seconde pour éviter un rattrapage excessif après un blocage. Sous forte charge, le jeu ralentit volontairement.

La pause empêche d'appeler `step`, puis remet l'accumulateur à zéro. Les compteurs de grâce et de dépassement n'avancent donc pas pendant un menu. F2 multiplie le temps à simuler par 0,25 sans changer le pas de calcul.

5. Repère et physique

Le repère est celui de l'écran : x augmente vers la droite, y vers le bas. Un objet en chute a une vitesse verticale positive. Son sommet se trouve à `y - rayon` ; il dépasse donc la limite si `y - rayon < limite`.

Les grandeurs utilisent des unités de simulation : positions en pixels, temps en secondes, vitesse en pixels/seconde, gravité en pixels/seconde². Les masses sont relatives, sans prétendre décrire les masses réelles des composants électroniques.

Pour une chute sans frottement, la mécanique continue donne :

- $v(t) = v0 + g \times t$;
- $y(t) = y0 + v0 \times t + g \times t^2 / 2$.

Un calcul numérique utilise de petits intervalles ; il approche ces formules. L'expérience `experiences/chute_libre.py` compare plusieurs fréquences. Elle retire obstacles et frottements pour isoler l'effet du pas de temps. Pymunk ne réemploie pas forcément la même variante d'Euler qu'un code écrit par les élèves : l'important est de mesurer et expliquer l'écart.

Paramètres à expérimenter

Paramètre	Valeur initiale	Effet à étudier
`GRAVITY`	1080	Durée de chute et vitesse d'impact
`ELASTICITY`	0,38	Intensité des rebonds
`FRICTION`	0,72	Glissement et rotations
`AIR_DAMPING`	0,985	Dissipation de vitesse en une seconde
`FIXED_DT`	1/120	Précision et coût de la simulation
`DROP_INTERVAL`	0,38	Rythme de placement
`DROP_GRACE`	1,15	Tolérance après création
`OVERFLOW_DELAY`	2,0	Dépassement continu autorisé

Pymunk multiplie les restitutions des deux formes qui se touchent : deux objets réglés à 0,38 ont une restitution de contact de 0,1444. Il multiplie également leurs coefficients de frottement. Ne pas confondre le coefficient d'un objet avec celui du contact.

La masse est proportionnelle à l'aire : `(rayon / rayon_initial)²`. Le moment d'inertie est obtenu par `pymunk.moment_for_circle`. Les gros objets repoussent donc davantage les petits et tournent selon leur inertie.

6. Organiser une fusion sûre

Pymunk détecte les contacts. Le callback `_on_contact` compare les niveaux et ajoute seulement les identifiants d'une paire candidate à un ensemble. Il ne retire pas les corps pendant le calcul physique.

Une fois `space.step` terminé, `_process_merges` :

1. Trie les paires par identifiant pour un choix déterministe.
2. Vérifie que les deux pièces existent encore et ont le même niveau.
3. Calcule une position et une vitesse moyennes pondérées par les masses.
4. Retire les deux anciennes pièces.
5. Crée le niveau suivant, ou retire les deux objets terminaux.
6. Ajoute le score et un événement d'animation / de son.

La position du nouvel objet est bornée par les parois et le sol. Son rayon plus grand peut recouvrir un voisin : Pymunk corrigera le chevauchement. Le jeu ne simule pas une fusion physiquement conservatrice : la nouvelle masse résulte du nouveau rayon, et les anciennes masses ne sont pas forcément additionnées. C'est un choix de jouabilité.

Trois objets qui se touchent

Supposons que les contacts soient `(1,2)`, `(1,3)` et `(2,3)`. Après la fusion de 1 et 2, ces deux identifiants sont retirés du dictionnaire. Les deux paires suivantes contiennent une pièce disparue : elles sont ignorées. La pièce 3 reste donc seule, et un nouvel objet reçoit un autre identifiant.

Le callback est `pre_solve` plutôt que seulement `begin` : il est rappelé tant que le contact existe. Une nouvelle pièce encore protégée par les 0,06 seconde de `MERGE_GRACE` pourra fusionner dès que cette grâce expire.

Invariants utiles

- Les identifiants des pièces présentes sont uniques.
- Une pièce supprimée ne reste dans aucun index.
- Une fusion normale retire deux pièces et en crée une : l'effectif diminue de 1.
- Deux objets terminaux retirés diminuent l'effectif de 2.
- Une pièce est consommée au plus une fois pendant la résolution d'un pas.
- Le score n'est ajouté qu'après une fusion effectivement réalisée.

7. Défaite et chronomètres

Un compteur global de débordement donnerait une règle injuste : une pièce dépassant une seconde, suivie d'une autre dépassant une seconde, provoquerait une défaite alors qu'aucune n'a dépassé deux secondes de suite.

Chaque pièce possède donc `overflow_time`. À chaque pas, si elle est assez ancienne et son sommet dépasse la limite, on ajoute `dt`. Sinon, on remet son compteur à zéro. La partie se termine si un compteur atteint le délai.

La tolérance commence à la création de la pièce, lors d'un lâcher ou d'une fusion. Après une fusion, la pièce possède un nouveau compteur. Le jeu prévoit aussi une fin de partie si une pièce s'échappe du bac, afin de ne pas laisser un objet hors de l'écran poursuivre une simulation infinie.

8. File et hasard reproductible

`current_level` désigne l'objet à lâcher et `next_level` celui annoncé. Après un lâcher réussi, le prochain devient le courant puis un nouveau prochain est tiré. Un clic trop rapide ne doit changer ni la file ni le score.

Les poids initiaux sont 32, 27, 20, 14 et 7, dont la somme vaut 100. Les probabilités correspondent donc à ces pourcentages. Il ne s'agit pas d'une distribution uniforme.

Une instance `random.Random(seed)` appartient au moteur. Les décorations utilisent un autre générateur : changer le nombre de particules ne doit pas changer la suite des objets. Avec `--fresh --seed 42`, comparer des paramètres en conservant les mêmes tirages. Il faut aussi conserver la même suite d'actions ; une graine ne reproduit pas les mouvements de la souris.

9. Événements et animations

Le moteur émet des `GameEvent` : `drop`, `impact`, `merge`, `goal`, `game_over`. L'application transmet ces messages à la vue et au son. Un événement de fusion contient une position, un niveau, des points et une indication de fusions rapprochées.

Les particules sont de petits points sans corps Pymunk. Elles n'encombrent pas les collisions. Les sprites sont mis en cache : un composant déjà dessiné est réutilisé. Pour créer un nouveau thème, modifier `art.py` et le catalogue, sans modifier la fusion.

Les notes sont des sinusoïdes échantillonnées. L'enveloppe diminue leur volume au fil du temps. Cela permet d'étudier une fonction mathématique et sa représentation numérique sans installer de bibliothèque audio supplémentaire.

10. Sauvegarder des objets complexes

Un corps Pymunk contient des objets natifs qui ne sont pas directement du JSON. `snapshot` en extrait des valeurs simples : niveaux, vecteurs, angles, compteurs et état du générateur aléatoire.

`from_snapshot` crée de nouveaux corps à partir de ces valeurs. JSON transforme les tuples en listes : il faut reconstruire des tuples pour `Random.setstate`. Les champs numériques sont vérifiés pour refuser NaN, les niveaux impossibles et les compteurs déraisonnables.

La sauvegarde est écrite dans un fichier temporaire, puis déplacée avec `os.replace`. Cette opération évite de remplacer l'ancien fichier avant que le nouveau soit entièrement écrit. La reprise recalcule les contacts : c'est une sauvegarde de partie, pas une simulation bit à bit identique.

11. Tester des comportements

Les tests fournis ne dessinent pas de captures comparées pixel par pixel. Ils vérifient les conséquences observables : un contact égal fusionne ; un contact différent ne fusionne pas ; trois objets ne sont pas consommés deux fois ; le prochain objet est respecté ; le sol arrête les chutes ; une même pièce doit dépasser continuellement ; la pause gèle le temps ; la sauvegarde conserve les prochains tirages.

Commande : `python -m unittest discover -s tests -v`.

Avant d'ajouter une fonctionnalité, écrire un scénario qui distingue le comportement attendu d'une erreur plausible. Exemple : un bouton placé à gauche ne doit pas lâcher un composant. Ce test protège la frontière entre entrées de l'interface et actions dans le bac.

Le test de charge lance plusieurs parties avec des positions pseudoaléatoires et contrôle les coordonnées finies et l'absence de traversée du sol. Il complète les tests isolés sans garantir tous les cas possibles de simulation.

12. Complexité et pistes de prolongement

Une détection naïve de tous les contacts entre n objets examine `n × (n - 1) / 2` paires, donc $O(n^2)$. Pymunk utilise des structures spatiales pour limiter les candidats ; son coût réel dépend de la disposition des corps. Ne pas lui attribuer une complexité constante garantie.

Pour k contacts candidats, le tri des paires coûte O(k log k). Les accès aux dictionnaires sont en moyenne O(1). La recherche de la pièce sous la souris parcourt les n objets : O(n). La projection verticale de visée est également O(n) à chaque image.

Prolongements pertinents : thème personnalisable, autre distribution de tirages, statistiques de partie en CSV, réglages de difficulté, import de sprites, chronologie des actions, meilleur guide de visée, puis moteur simplifié écrit par les élèves.

Un mode duel ou une version Android demande un travail supplémentaire. Pour un premier projet, privilégier une amélioration mesurable et documentée du jeu local avant d'ajouter des services ou une nouvelle plateforme.

Projet NSI — Circuit Merge : assemblez l'informatique

Mission

Créer un jeu Python où le joueur lâche des composants électroniques dans un bac. Deux composants du même niveau fusionnent au contact en un objet plus grand. Le joueur cherche à atteindre le supercalculateur tout en évitant le débordement.

Le projet se réalise en binôme ou trinôme. Proposition : **8 séances de 1 h 30**, soit 12 heures, avec adaptation possible à 6–10 séances selon l'expérience des élèves. Le professeur peut fournir le moteur physique et faire construire les règles / l'interface, ou proposer une version plus ambitieuse en terminale.

Contrat minimal

1. Une fenêtre contient un bac ouvert et un composant à lâcher.
2. La souris ou le clavier permet de choisir la position horizontale.
3. Le composant tombe, rebondit et s'appuie sur le sol / les autres objets.
4. Deux objets de même niveau fusionnent en un seul du niveau suivant.
5. Le score augmente après une fusion ; le prochain objet est annoncé.
6. Une pièce dépassant trop longtemps la ligne provoque une fin de partie.
7. Le joueur peut recommencer et mettre en pause.
8. Le code comporte fonctions, noms explicites et documentation des choix.

Commencer avec quatre niveaux et des cercles colorés. Ajouter les onze niveaux et les illustrations après validation des règles. Le mode pédagogique et la sauvegarde constituent des prolongements, pas des prérequis au premier prototype.

Progression par séances

Séance	Travail attendu	Preuve de réussite
1	Décrire les règles, dessiner le repère, compléter les fonctions simples	Schéma, contrats et cas limites
2	Afficher bac / objet, lire les entrées, borner la visée	Aucun lâcher ne commence dans une paroi
3	Ajouter gravité, sol et collisions avec Pymunk	Deux objets différents s'empilent
4	Séparer données et règles, organiser la file de fusion	Trois objets ne donnent pas deux fusions incohérentes
5	Ajouter score, prochain objet et tirage pondéré	Le composant annoncé est bien le prochain lâché
6	Défaite, pause, reprise et nouvelle partie	Un menu n'avance pas le chronomètre de défaite
7	Améliorer le thème, ajouter une extension et des tests	Comparaison avant/après avec un critère mesuré
8	Finaliser documentation, démonstration et présentation orale	Projet lancé sur un autre ordinateur

Répartition du travail

En binôme : une personne travaille sur le moteur et les règles, l'autre sur les entrées / le rendu ; échanger les rôles au milieu du projet. En trinôme, une troisième responsabilité regroupe les tests, les données et la persistance.

Chaque élève doit pouvoir expliquer une fonction écrite par un partenaire. Se mettre d'accord sur les noms des fonctions, les arguments, les valeurs renvoyées et les événements avant de travailler séparément. Les rôles ne dispensent personne de comprendre le jeu complet.

Travail de découverte sans bibliothèque

Compléter `atelier_nsi/regles_a_completer.py` : borner une abscisse, reconnaître une fusion possible, calculer des points, détecter un dépassement et choisir des paires sans réutiliser une pièce.

Pour chaque fonction, proposer au moins un cas ordinaire et un cas limite. Justifier le résultat attendu. Le squelette est autonome : il prépare le raisonnement, puis les fonctions pourront être adaptées au moteur du groupe.

Expérience obligatoire

Choisir **un seul paramètre** parmi gravité, restitution, frottement ou pas de temps. Formuler une hypothèse avant de modifier le programme. Garder une graine de tirages fixe et un scénario d'actions aussi identique que possible.

Présenter un petit tableau avec le paramètre, deux valeurs et une mesure concrète : temps de chute, nombre de rebonds, durée d'une simulation, etc. Expliquer les limites de la comparaison. Le score seul dépend trop du joueur pour établir facilement l'effet d'un paramètre physique.

Livrables

- Dossier contenant le programme, les dépendances et les ressources nécessaires.
- README indiquant installation, lancement, contrôles et fonctionnalités.
- Schéma de l'organisation des modules et description de deux algorithmes.
- Quelques tests utiles et leurs résultats, avec les cas qu'ils protègent.
- Tableau de l'expérience et interprétation des résultats.
- Présentation orale de 5–7 minutes, démonstration incluse.
- Une trace des contributions de chaque élève (journal, commits ou tableau).

Proposition de barème sur 20

Critère	Points	Indicateurs
Règles fonctionnelles	6	Chute / collisions 2, fusion 2, file et score 1, fin de partie / reprise 1
Structure et qualité du code	4	Fonctions / modules 2, noms et contrats 1, responsabilités 1
Tests et cas limites	3	Scénarios pertinents 2, explication d'un bug évité 1
Démarche expérimentale	2	Hypothèse / protocole 1, mesures / interprétation 1
Documentation et autonomie de lancement	2	README 1, explications / dépendances 1
Présentation et maîtrise individuelle	3	Démonstration 1, explication 1, questions 1

Une extension esthétique seule ne compense pas une fusion incorrecte. Le barème peut récompenser l'ambition dans les rubriques concernées tout en gardant la priorité à un programme compris et fiable.

Questions pour l'oral

- Pourquoi distinguer le niveau et l'identifiant d'un composant ?
- Que se passe-t-il si trois objets identiques se touchent au même instant ?
- Pourquoi ne pas faire évoluer la physique d'une distance fixe par image ?

- Comment garantir que le prochain objet annoncé est réellement utilisé ?
- Pourquoi le compteur de dépassement appartient-il à chaque pièce ?
- Quelles données faut-il sauvegarder pour reprendre la partie ?
- Quel test a détecté ou empêché une erreur dans votre projet ?

Extensions au choix

Préférer une extension aboutie : statistiques CSV, niveaux de difficulté, thème graphique remplaçable, panneau d'inspection, sauvegarde JSON, comparaison de distributions de tirages ou classement local.

Les fonctionnalités ajoutées doivent être décrites dans le README et accompagnées d'un scénario montrant qu'elles fonctionnent.

Validation — version 1.0.0

Vérifications exécutées le **6 octobre 2026**.

Environnement testé

- Python 3.12.14, Linux x86-64.
- Pygame 2.6.1, Pymunk 7.3.0.
- SDL en mode sans écran pour les scénarios automatisés et les captures.

Résultats

- **20 tests automatisés réussis** avec `python -m unittest discover -s tests -v`.
- Test de lancement sur 120 images réussi avec `--smoke-test 120 --mute`.
- Compilation syntaxique des modules, expériences et exercices réussie.
- Capture du montage de démonstration générée avec le vrai rendu Pygame.
- Inspection visuelle de l'interface, des textes, du bac et des composants.
- Expérience de chute libre exécutée à 30, 60, 120 et 240 Hz : les écarts de position après une seconde valent 18, 9, 4,5 et 2,25 pixels pour le modèle testé.

Comportements couverts

Contact égal / inégal, trois contacts simultanés, fusion en chaîne, paire terminale, prochaine pièce, délai de lâcher, bornes de visée, arrêt au sol, stabilité de plusieurs parties pseudoaléatoires, tolérance de création, dépassement continu, chronomètres séparés, fin de partie, sauvegarde JSON, tirages restaurés, rejet de NaN, pause, clic / relâcher, clic sur un menu, clavier, mode NSI, ralenti, perte de focus et redimensionnement.

Portée

Les essais de simulation et de rendu ont été effectués sous Linux. Les scripts Windows et les instructions macOS sont fournis mais n'ont pas été exécutés sur ces systèmes. Le son a été initialisé avec le pilote SDL simulé, sans écoute sur un périphérique réel. Le plein écran et le toucher matériel n'ont pas été vérifiés sur un appareil physique.

Les contacts au chargement sont recalculés : un état sauvegardé ne garantit pas une reprise identique bit à bit. Les tests de charge ne couvrent pas tous les empilements possibles. Les paramètres sont ceux de Circuit Merge ; ils n'ont pas été extraits du jeu Android.

Références

Fruit Merge: Match Game — fiche du jeu
Pymunk — documentation officielle
Pygame — documentation officielle